

Natural Language Processing

7. Sequence processing with recurrent networks

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Recurrent Neural Networks
- Applications
- Fancier Architectures
- Vanishing Gradient

Recurrent Neural Networks

The Temporal Nature of Language

- Language is inherently sequential
 - *Flow* of conversations, news *feeds*, twitter streams
- Machine Learning approaches seen for text classification
 - LR, MLP: **simultaneous access** to all aspects of input
 - Fixed-size input vectors capturing all relevant aspects at once
 - No temporal nature
- Language modeling work-around
 - Fixed-size window of tokens as input
 - Longer sequences: **sliding window**, sequence of predictions
 - Decision made for one window no impact on later predictions

Neural Language Modeling

output distribution

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|}$$

hidden layer

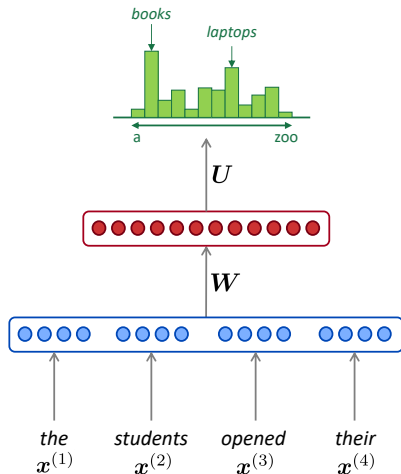
$$h = f(We + b_1)$$

concatenated word embeddings

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

words / one-hot vectors

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



A Fixed-Window Neural Language Model

Improvement over n-gram Language Models:

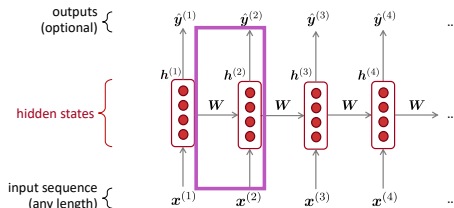
- No sparsity problem
- No need to store all observed n-grams

Problems with sliding window:

- **Fixed context size**
 - Limit of all Markov approaches
 - Many language tasks require access to arbitrarily distant information
 - Window is **too small**: enlarging it enlarges W
- **Difficult to learn systematic patterns** arising from phenomena like constituency
 - Phrases appearing in separate windows in consecutive processing steps
 - *the students*: first $x^{(3)}x^{(4)}$ then $x^{(2)}x^{(3)}$ then $x^{(1)}x^{(2)}$
 - **No symmetry** in how the inputs are processed

Recurrent Neural Networks

Idea: **apply the same weights W repeatedly**



- A **Recurrent Neural Network** is a network containing cycles
- The value of a unit depends on earlier outputs as an input
- Powerful but difficult to reason with and train
- Constrained architectures are effective

Simple Recurrent Neural Networks (Vanilla, Elman)

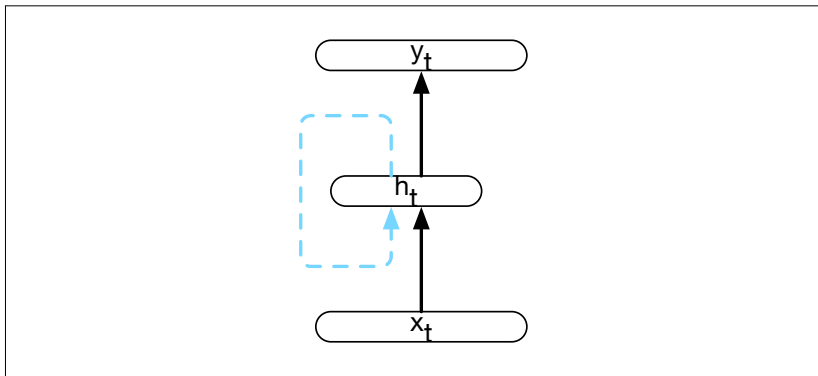


Figure 9.2 Simple recurrent neural network after Elman (Elman, 1990). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

Recurrent Neural Networks

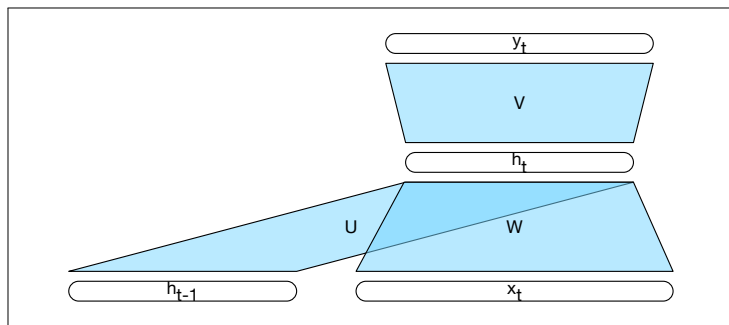


Figure 9.3 Simple recurrent neural network illustrated as a feedforward network.

$$h_t = g(Uh_{t-1} + Wx_t)$$

$$y_t = f(Vh_t)$$

An initial h_0 is given. Usually, f is **softmax**.

Recurrent Neural Networks

- The network is presented **one element at a time**
- Input vector x_t multiplied by a weight matrix W and passed through a (non-linear) activation function g to obtain activation value for hidden units h_t
- Hidden unit h_t is also fed a **recurrent link** from h_{t-1}
- h_{t-1} provides a form of **memory** or **context**, encoding earlier processing
- **No fixed limit on prior context**
- **Model size doesn't increase** for longer input
- **Symmetry**: same weights applied on every time step

Elman Network Unrolled

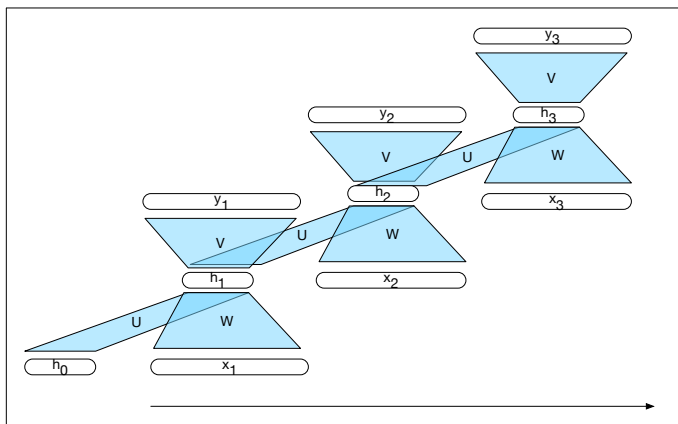


Figure 9.5 A simple recurrent neural network shown unrolled in time. Network layers are copied for each time step, while the weights U , V and W are shared in common across all time steps.

Different values over time; same weights

Neural Language Modeling

output distribution

$$\hat{y}^{(t)} = \text{softmax}(Uh^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

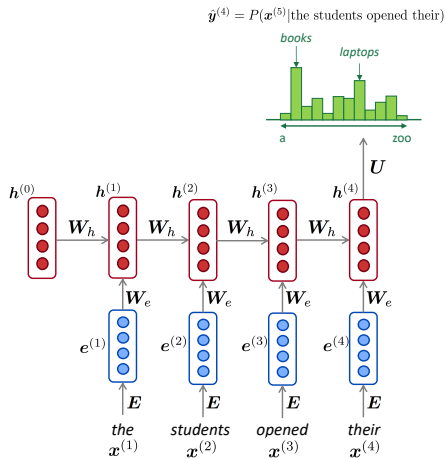
$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

word embeddings

$$e^{(t)} = Ex^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Training an RNN Language Model

- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output dist $\hat{y}^{(t)}$ **for every step** t
 - i.e., predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability dist $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$)

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = - \log \hat{y}_{x_{t+1}}^{(t)}$$

- Average this to get **overall loss** for entire training set

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{y}_{x_{t+1}}^{(t)}$$

Training an RNN Language Model

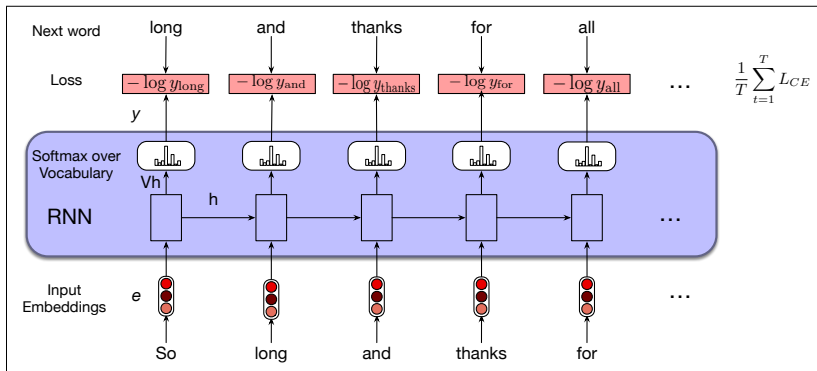


Figure 9.6 Training RNNs as language models.

To always give the model the correct word: **teacher forcing**

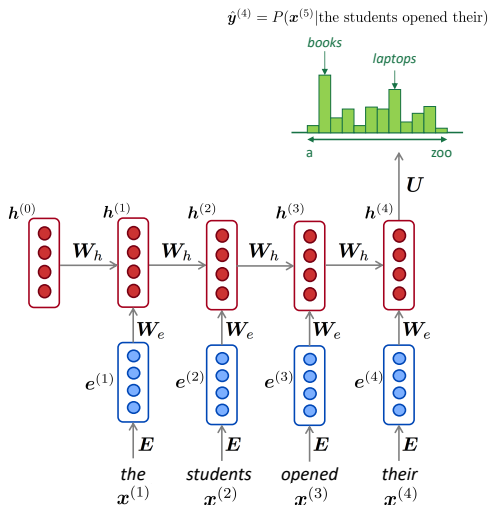
Training an RNN Language Model

Predicted probability distributions $\hat{y}^{(t)}$

$$\Rightarrow J^{(t)}(\theta) = -\log \hat{y}^{(t+1)}$$

Iterate through the entire corpus $x^{(1)}, x^{(2)}, \dots$

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$



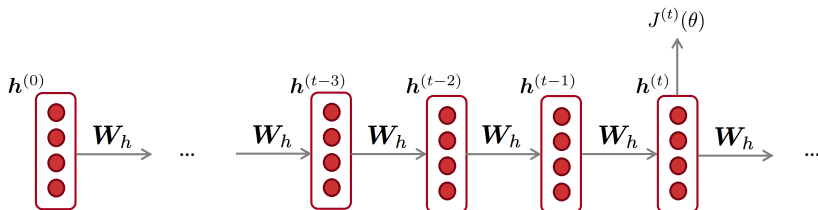
Training an RNN Language Model

- However, computing loss and gradients across the **entire corpus** $x^{(1)}, \dots, x^{(T)}$ is **too expensive**

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $x^{(1)}, \dots, x^{(T)}$ as a **sentence** (or a **document**)
- Recall: **Stochastic Gradient Descent** allows us to compute loss and gradients for small chunk of data, and update
- Compute loss $J(\theta)$ for a sentence (actually a batch of sentences), compute gradients and update weights. Repeat.

Backpropagation for Recurrent Neural Networks



Backpropagation through time: Backpropagate over timesteps $i = t, \dots, 0$, summing gradients as you go.

RNN Applications

RNN for Autoregressive Generation

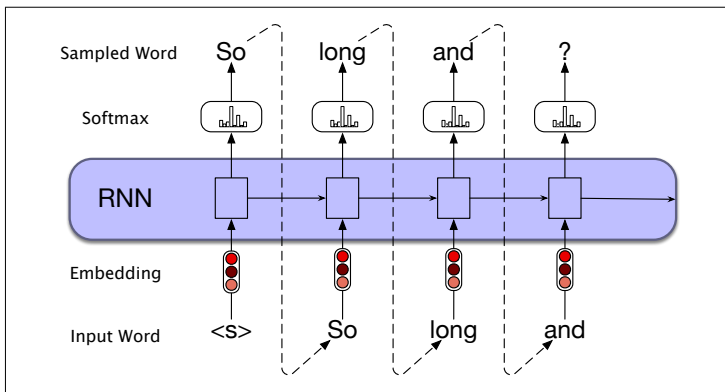


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

Just like an n-gram Language Model, you can use an RNN Language Model to **generate text by repeated sampling**.

RNN for Autoregressive Generation

- Shannon's method in a neural context:
 - Use <s> as first input
 - Sampled output is next step's input
 - Continue until end-of-sentence marker </s> is produced, or fixed length limit is reached
- **Autoregressive generation**: word generated at each time step is **conditioned** on word generated at previous step
- Notice the different input between test time (autoregressive generation) and training time (**teacher forcing**)
- Key to state-of-the-art approaches to NLG applications is to prime the generation component with an appropriate context

RNN for Autoregressive Generation

🚲 For fun: train a RNN-LM on any kind of text, then generate text in that style.

- Obama speeches
- Harry Potter
- Recipes
- Paint color names (character-level RNN-LM)

RNNs for Sequence Labeling

E.g., **POS-tagging**, named-entity recognition, structure prediction

- In combination with CRF to select most likely tag sequence

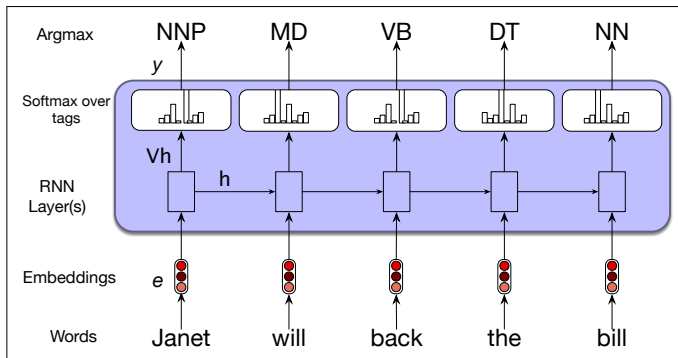


Figure 9.7 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

RNN for Sequence Classification

E.g., sentiment analysis

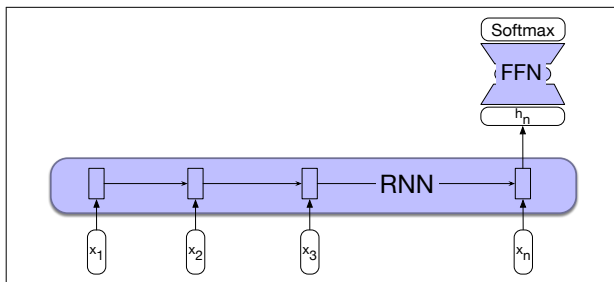


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

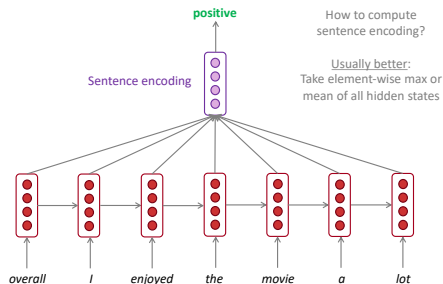
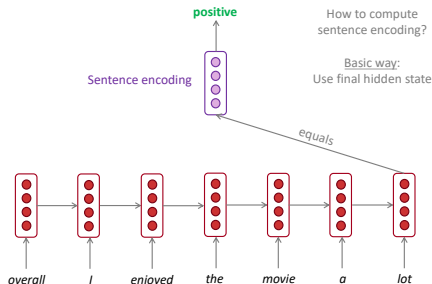
RNN for Sequence Classification

E.g., **sentiment analysis**, document-topic classification, spam detection, message routing, deception detection, ...

- RNN used to produce **sentence encoding**
- Training driven by output of softmax
- Error signal backpropagated all the way through the classifier and RNN
- Deep neural network with **end-to-end training**

Sentence Encoding

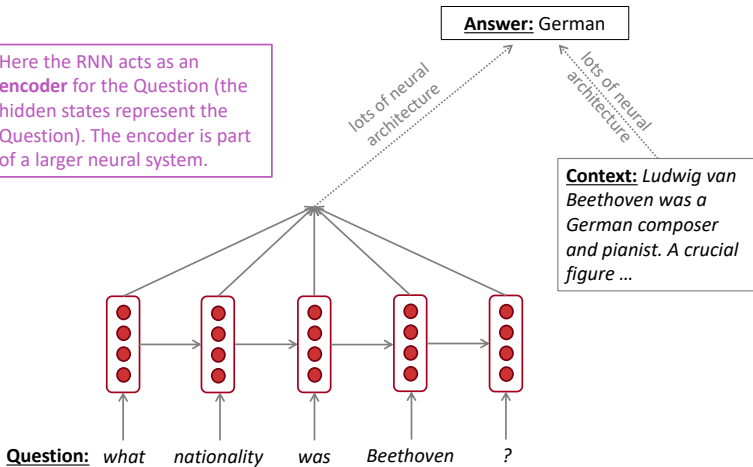
Sentence encoding could be computed as the **final hidden state** or as element-wise max or mean of **all hidden states**



RNN as an Encoder Module

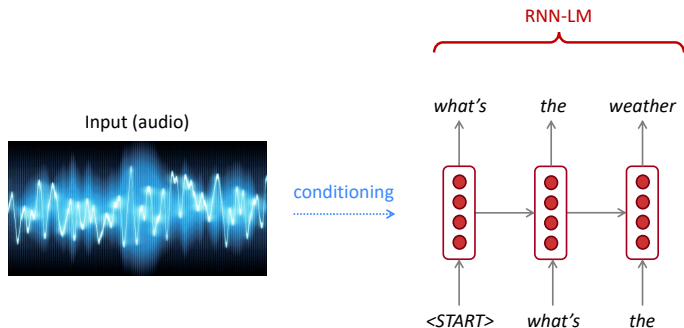
E.g., **question answering**, machine translation, ...

Here the RNN acts as an **encoder** for the Question (the hidden states represent the Question). The encoder is part of a larger neural system.



RNN for Text Generation

E.g., **speech recognition**, machine translation, summarization, ...



Conditional language model

Fancier Architectures

Stacked (Multi-Layer) RNN

The sequence of outputs from one RNN used as input to another RNN

- Induce representations at different levels of abstraction
- However, **training cost rises steeply** with number of levels

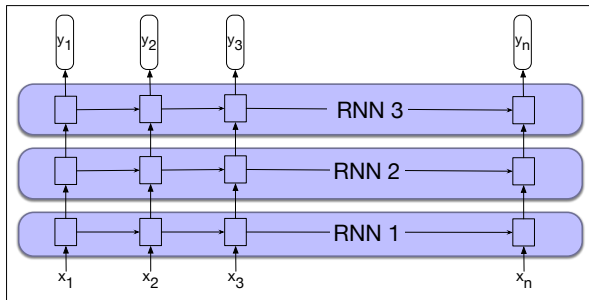


Figure 9.10 Stacked recurrent networks. The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.

Bidirectional RNN

Useful when entire input sequence available all at once (**not** for LM)

- Exploit **context to the right** of current input as well
- Two independent RNNs; outputs usually concatenated

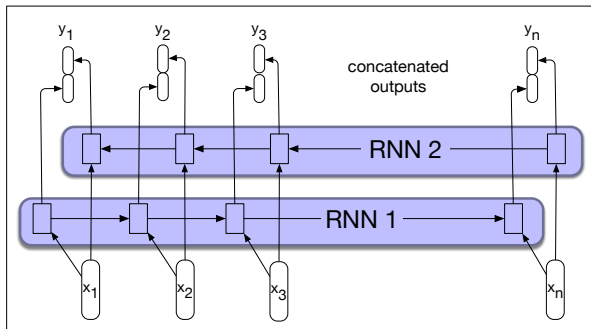


Figure 9.11 A bidirectional RNN. Separate models are trained in the forward and backward directions, with the output of each model at each time point concatenated to represent the bidirectional state at that time point.

Bidirectional RNN for Sequence Classification

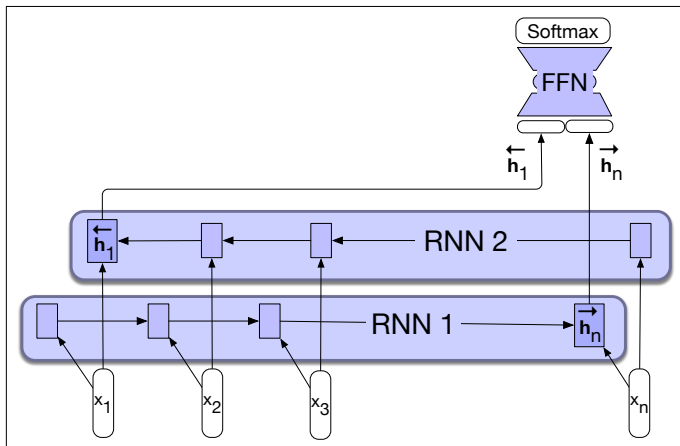


Figure 9.12 A bidirectional RNN for sequence classification. The final hidden units from the forward and backward passes are combined to represent the entire sequence. This combined representation serves as input to the subsequent classifier.

Vanishing Gradient

Vanishing Gradient

- Gradient can be viewed as a measure of the effect of the past on the future
- Gradient signal from faraway is lost because it's much smaller than gradient signal from close-by
- Model weights are updated only with respect to near effects, not long-term effects
- If the gradient becomes vanishingly small over longer distances (step t to step $t + n$), then we can't tell whether:
 - There's no dependency between step t and $t + n$ in the data
 - We have wrong parameters to capture the true dependency between t and $t + n$

Effect of Vanishing Gradient on RNN-LM

- **LM task:** *The writer of the books* ____
 - **Syntactic** recency: The writer of the books *is*
 - **Sequential** recency: The writer of the books *are*
- Vanishing gradient problems may bias RNN-LMs towards learning from sequential recency, so they make this type of error more often than we'd like
- It's too difficult for the RNN to learn to preserve information over many timesteps

Long Short-Term Memory (LSTM) Networks

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
- On step t , there is a **hidden state** $h^{(t)}$ and a **cell state** $c^{(t)}$
 - Both are vectors of length n
 - The cell stores **long-term information**
 - The LSTM can **erase**, **write** and **read** information from the cell
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
 - The gates are also vectors of length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
 - The gates are **dynamic**: their value is computed based on the current context

Long Short-Term Memory (LSTM) Networks

Gates:

- **Forget gate:** controls what is kept vs forgotten, from previous cell state

$$f^{(t)} = \sigma \left(W_f h^{(t-1)} + U_f x^{(t)} + b_f \right)$$

- **Input gate:** controls what parts of the new cell content are written to cell

$$i^{(t)} = \sigma \left(W_i h^{(t-1)} + U_i x^{(t)} + b_i \right)$$

- **Output gate:** controls what parts of cell are output to hidden state

$$o^{(t)} = \sigma \left(W_o h^{(t-1)} + U_o x^{(t)} + b_o \right)$$

Long Short-Term Memory (LSTM) Networks

New state:

- **New cell content:** this is the new content to be written to the cell

$$\tilde{c}^{(t)} = \tanh \left(W_c h^{(t-1)} + U_c x^{(t)} + b_c \right)$$

- **Cell state:** *erase* some content from last cell state (forget), and write some *new* cell content (input)

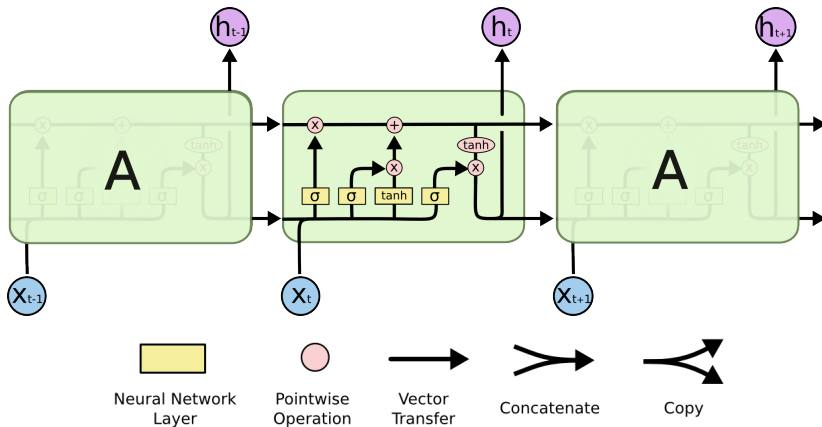
$$c^{(t)} = f^{(t)} \cdot c^{(t-1)} + i^{(t)} \cdot \tilde{c}^{(t)}$$

- **Hidden state:** read some content from the cell (output)

$$h^{(t)} = o^{(t)} \cdot \tanh c^{(t)}$$

Long Short-Term Memory (LSTM) Networks

Visually:



Understanding LSTMs, colah's blog

Long Short-Term Memory (LSTM) Networks

A different illustration of LSTM:

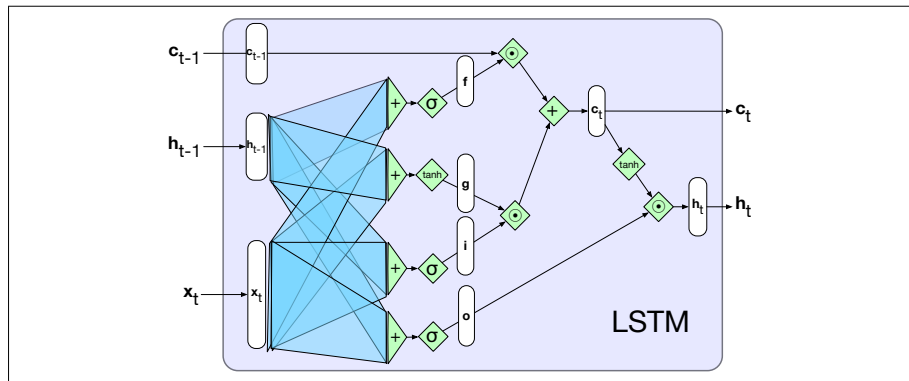


Figure 9.13 A single LSTM unit displayed as a computation graph. The inputs to each unit consists of the current input, x , the previous hidden state, h_{t-1} , and the previous context, c_{t-1} . The outputs are a new hidden state, h_t and an updated context, c_t .

Long Short-Term Memory (LSTM) Networks

- The LSTM architecture makes it **easier** for the RNN to **preserve information over many timesteps**
 - e.g. if the forget gate is set to 1 for a cell dimension and the input gate set to 0, then the information of that cell is preserved indefinitely
 - By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state
- LSTM doesn't *guarantee* that there is no vanishing gradient, but it does provide an easier way for the model to learn long-distance dependencies
- Dominant approach in 2013-2015 with state-of-the-art results in handwriting recognition, speech recognition, machine translation, parsing, image captioning, ...

Gated Recurrent Units (GRU)

- Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM
- On each timestep t : input $x^{(t)}$ and hidden state $h^{(t)}$ (**no cell state**)
- Update gate**: controls what parts of hidden state are updated vs preserved

$$u^{(t)} = \sigma \left(W_u h^{(t-1)} + U_u x^{(t)} + b_u \right)$$

- Reset gate**: controls what parts of previous hidden state are used to compute new content

$$r^{(t)} = \sigma \left(W_r h^{(t-1)} + U_r x^{(t)} + b_r \right)$$

Gated Recurrent Units (GRU)

- Proposed by Cho et al. in 2014 as a simpler alternative to the LSTM
- On each timestep t : input $x^{(t)}$ and hidden state $h^{(t)}$ (**no cell state**)
- **Update gate**: what parts of hidden state are updated
- **Reset gate**: what parts of previous hidden state into new content
- **New hidden state content**: *reset gate* selects useful parts of previous hidden state. Use this and current input to compute new hidden content

$$\tilde{h}^{(t)} = \tanh \left(W_h(r^{(t)} \cdot h^{(t-1)}) + U_h x^{(t)} + b_h \right)$$

- **Hidden state**: *update gate* simultaneously controls what is kept from previous hidden state, and what is updated to new hidden state content

$$h^{(t)} = (1 - u^{(t)}) \cdot h^{(t-1)} + u^{(t)} \cdot \tilde{h}^{(t)}$$

Skip Connections

- Vanishing/exploding gradient can be a problem for all (deep) neural architectures, not only RNNs
- Due to chain rule/choice of nonlinearity function, gradient can become vanishingly small as it backpropagates
- Thus lower layers are learnt very slowly (hard to train)
- Solution: allow gradient to flow by **adding more direct connections**
- Example: residual (skip), dense, highway connections

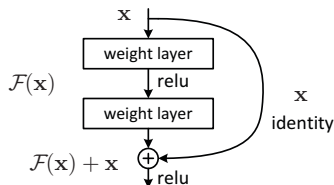


Figure 2. Residual learning: a building block.

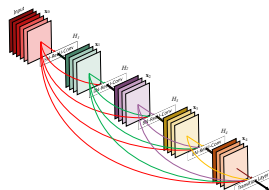


Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

Character-Level Word Embeddings

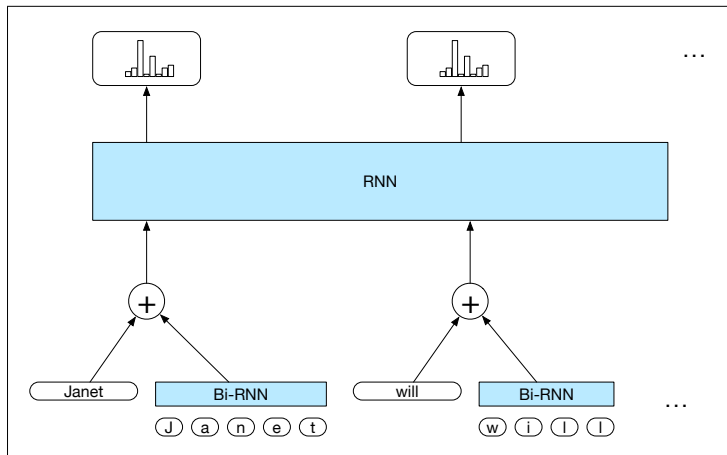


Figure 9.15 Sequence labeling RNN that accepts distributional word embeddings augmented with character-level word embeddings.

Character-Level Word Embeddings

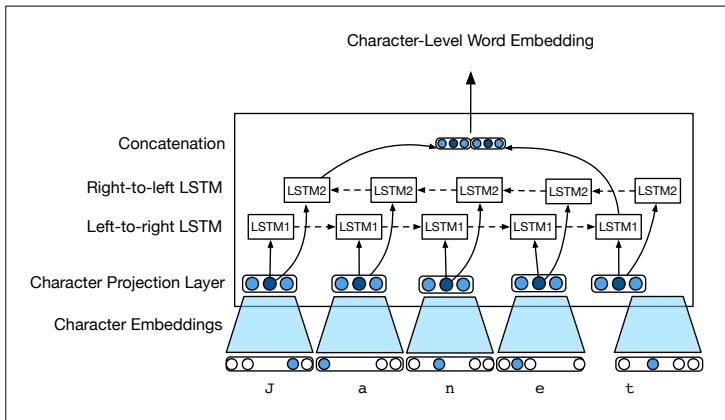


Figure 9.16 Bi-RNN accepts word character sequences and emits embeddings derived from a forward and backward pass over the sequence. The network itself is trained in the context of a larger end-application where the loss is propagated all the way through to the character vector embeddings.

Summary

- **RNN**: a family of neural networks that
 - take sequential input of **any length**,
 - apply **same weights on each step**, and
 - can optionally produce output on each step
- RNN can be used
 - for language modeling
 - for sequence labeling
 - for sentence classification
 - in encoder/decoder architectures
- **LSTMs** are powerful but **GRUs** are faster
- Use **bidirectionality** when possible
- **Stacked RNNs** are powerful but might need skip/dense connections

Questions?